

Ex.no:8	MINI PROJECT – Air Quality Analysis

Aim:

To clean, transform, store, and analyze hotel booking data using Python, MySQL, Power BI and Tableau to derive meaningful insights.

Algorithm:

1. Collect and clean the air quality dataset.
2. Select **20,000 records** and store them in SQL Server.
3. Perform data analysis using SQL queries.
4. Connect SQL Server to **Power BI**.
5. Create charts for pollutant levels, cities, and trends.
6. Combine the charts into an interactive dashboard.
7. Analyze the results and save the report.

Architecture:

PROGRAM

```
import pandas as pd
```

```
import pyodbc
```

```
# =====
```

```
# 1. PYTHON – DATA CLEANING & TRANSFORMATION
```

```
# =====
```

```
# Load dataset
```

```
df = pd.read_excel(
```

```
    "india_air_quality_consolidated.csv.xlsx"
```

```
)
```

```
# Create a copy for processing
```

```
air = df.copy()
```

```
# Remove duplicate records
```

```
air.drop_duplicates(inplace=True)
```

```
# Handle missing values
```

```
air["city"] = air["city"].fillna("Unknown")
```

```
air["location"] = air["location"].fillna("Unknown")
```

```
# Convert date column
```

```
air["date"] = pd.to_datetime(
```

```
    air["date"],
```

```
    errors="coerce"
```

```
)
```

```
# Convert pollutant columns to numeric
```

```
pollutants = [
```

```
    "pm25",
```

```
    "pm10",
```

```
    "o3",
```

```
    "no2",
```

```
    "so2",
```

```
    "co"
```

```
]
```

```
for column in pollutants:
```

```
    air[column] = pd.to_numeric(
```

```
        air[column],
```

```
        errors="coerce"
```

```
)
```

```
# Select 20,000 records
```

```
air = air.sample(  
    n=20000,  
    random_state=42  
)
```

```
# Standardize column names
```

```
air.columns = (  
    air.columns  
    .str.lower()  
    .str.strip()  
    .str.replace(" ", "_")  
)
```

```
print("Original Shape:", df.shape)
```

```
print("Cleaned Shape:", air.shape)
```

```
# =====
```

```
# 2. SQL SERVER – DATABASE CONNECTION
```

```
# =====
```

```
server = "localhost"
```

```
database = "AirQualityProject"
```

```
connection_string = (  
    "DRIVER={ODBC Driver 17 for SQL Server};"  
    f"SERVER={server};"  
    f"DATABASE={database};"  
    "Trusted_Connection=yes;"  
)
```

```
try:
    connection = pyodbc.connect(
        connection_string
    )

    print("\nConnected to SQL Server successfully!")

except Exception as e:
    print("\nSQL Server connection failed!")
    print(e)
    exit()

# =====
# 3. INSERT DATA INTO SQL SERVER
# =====

cursor = connection.cursor()

# Clear existing records
cursor.execute(
    "DELETE FROM AirQuality"
)

connection.commit()

# Insert query
insert_query = """
INSERT INTO AirQuality
(
    city,
```

```
location,  
date,  
pm25,  
pm10,  
o3,  
no2,  
so2,  
co  
)  
VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)  
""
```

```
# Insert records
```

```
for _, row in air.iterrows():
```

```
    cursor.execute(  
        insert_query,
```

```
        row["city"],  
        row["location"],  
        row["date"],
```

```
        row["pm25"]  
        if pd.notna(row["pm25"])  
        else None,
```

```
        row["pm10"]  
        if pd.notna(row["pm10"])  
        else None,
```

```
        row["o3"]  
        if pd.notna(row["o3"])
```

```

        else None,

        row["no2"]
        if pd.notna(row["no2"])
        else None,

        row["so2"]
        if pd.notna(row["so2"])
        else None,

        row["co"]
        if pd.notna(row["co"])
        else None
    )

connection.commit()

print("Data loaded successfully into SQL Server!")

# =====

# 4. VERIFY DATA

# =====

cursor.execute(
    "SELECT COUNT(*) FROM AirQuality"
)

total_records = cursor.fetchone()[0]

print(
    "\nTotal records in SQL Server:",

```

```

        total_records
    )

# =====

# 5. SQL ANALYSIS

# =====

print("\nAverage Pollutant Values")
print("-----")

query = """
SELECT
    AVG(pm25) AS Average_PM25,
    AVG(pm10) AS Average_PM10,
    AVG(o3) AS Average_O3,
    AVG(no2) AS Average_NO2,
    AVG(so2) AS Average_SO2,
    AVG(co) AS Average_CO
FROM AirQuality
"""

pollutant_result = pd.read_sql(
    query,
    connection
)

print(pollutant_result)

# =====

# 6. CITY-WISE PM2.5 ANALYSIS

```

```
# =====
```

```
print("\nAverage PM2.5 by City")
```

```
print("-----")
```

```
city_query = """
```

```
SELECT
```

```
    city,
```

```
    AVG(pm25) AS Average_PM25
```

```
FROM AirQuality
```

```
GROUP BY city
```

```
ORDER BY Average_PM25 DESC
```

```
"""
```

```
city_result = pd.read_sql(
```

```
    city_query,
```

```
    connection
```

```
)
```

```
print(city_result)
```

```
# =====
```

```
# 7. TOP 10 PM2.5 RECORDS
```

```
# =====
```

```
print("\nTop 10 PM2.5 Records")
```

```
print("-----")
```

```
top_query = """
```

```
SELECT TOP 10
```

```
    city,
```



```
        location,  
        date,  
        pm25,  
        pm10  
FROM AirQuality  
ORDER BY pm25 DESC  
""""
```

```
top_result = pd.read_sql(  
    top_query,  
    connection  
)
```

```
print(top_result)
```

```
# =====  
# 8. DATA RETRIEVAL FOR POWER BI  
# =====
```

```
powerbi_query = """
```

```
SELECT  
    city,  
    location,  
    date,  
    pm25,  
    pm10,  
    o3,  
    no2,  
    so2,  
    co  
FROM AirQuality
```

```
"""
```

```
powerbi_data = pd.read_sql(  
    powerbi_query,  
    connection  
)
```

```
print(  
    "\nData retrieved for Power BI:",  
    len(powerbi_data),  
    "records"  
)
```

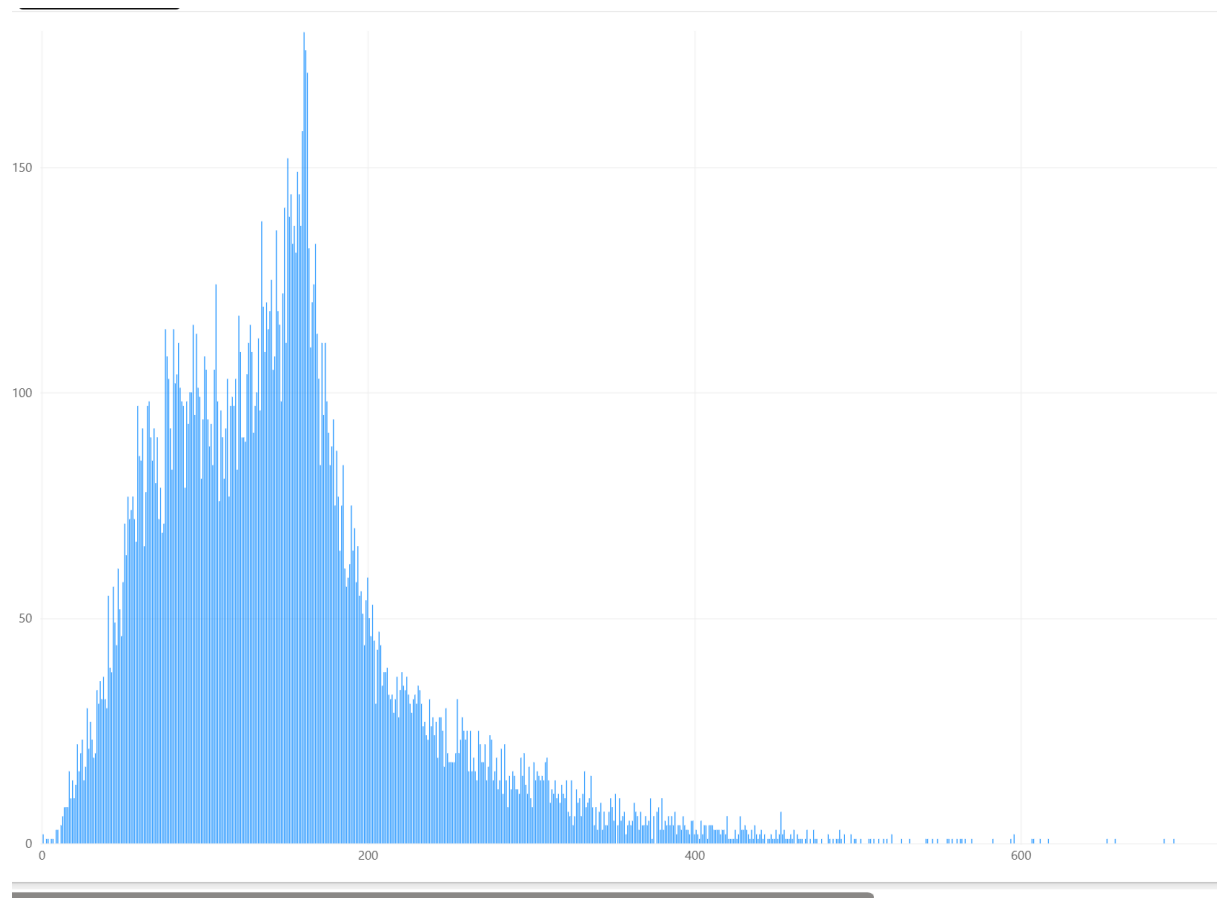
```
# =====  
# 9. CLOSE SQL SERVER CONNECTION  
# =====
```

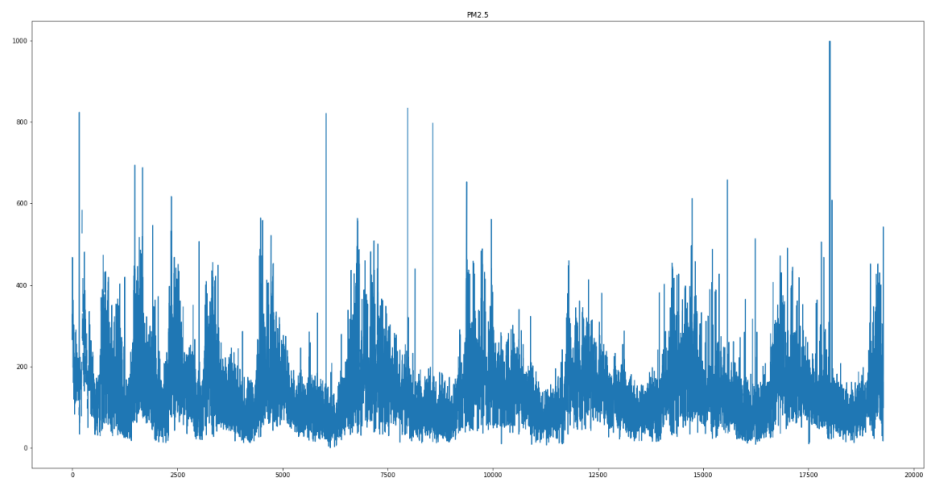
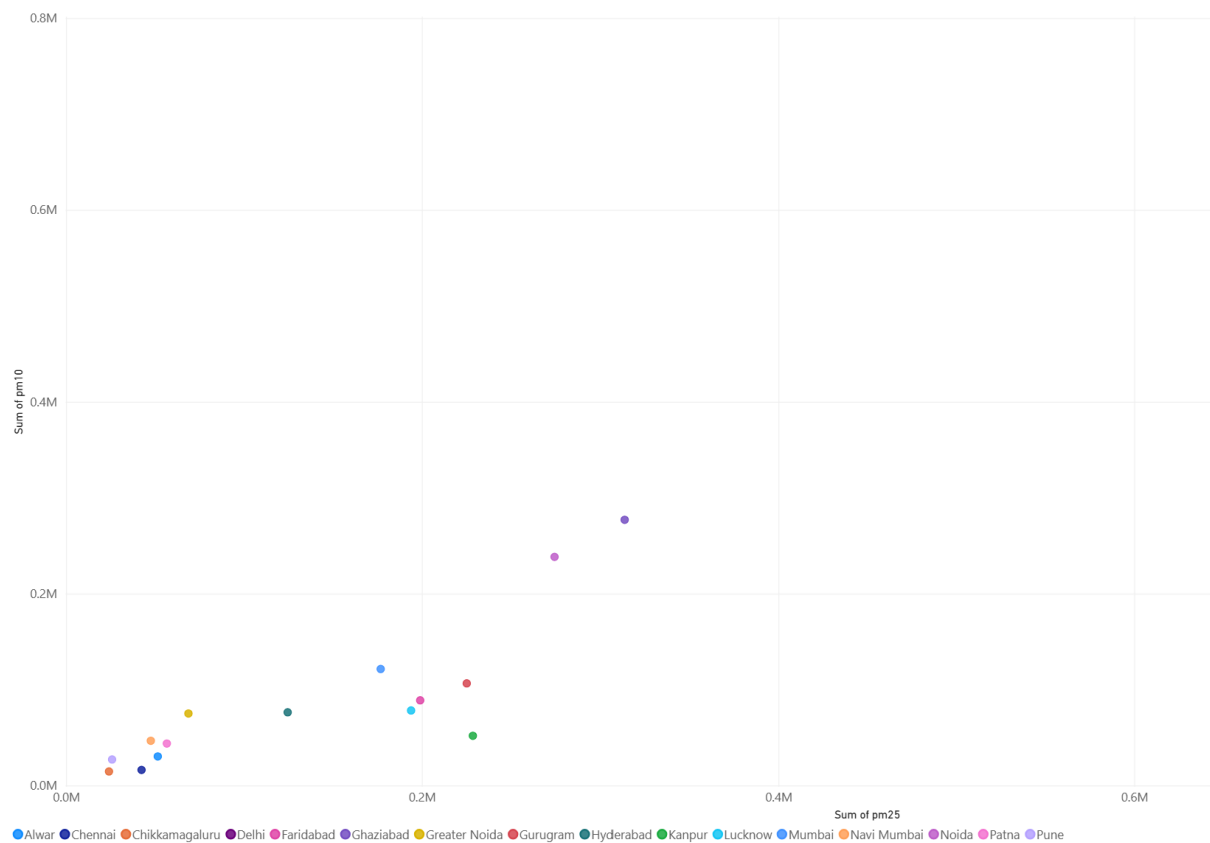
```
cursor.close()  
connection.close()
```

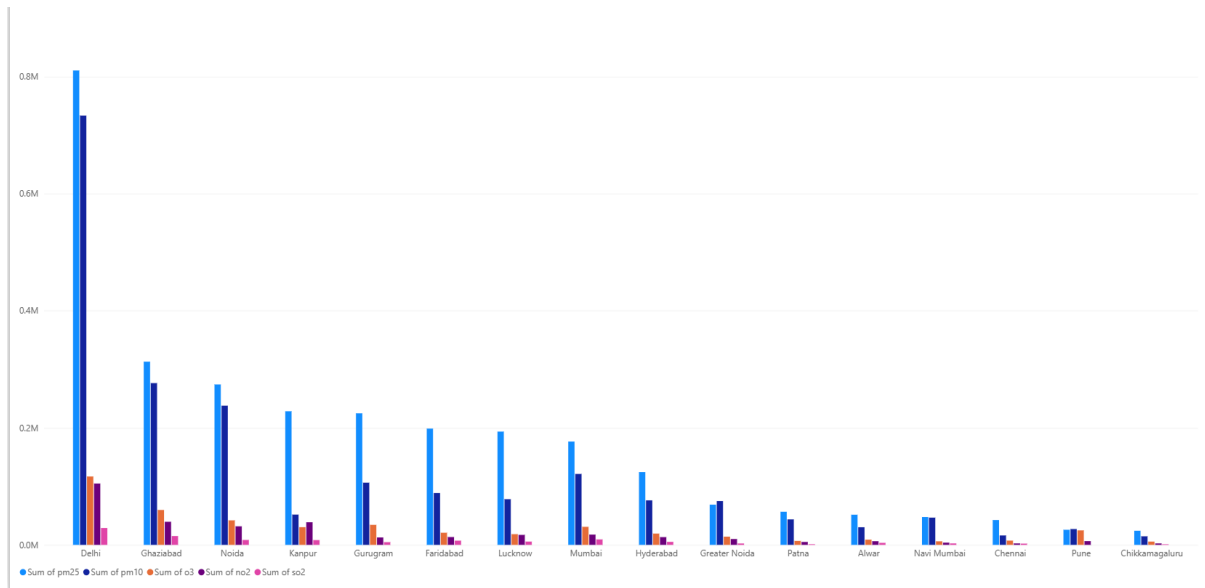
```
print("\nSQL Server connection closed.")
```

```
print("\n=====")  
print("PROJECT 1 COMPLETED SUCCESSFULLY")  
print("=====")
```

Output:







Result:

The Visualizations from Power BI is done successfully.

Ex.no:8	MINI PROJECT – Mobile Phone Sales Analysis

Aim:

To clean, transform, store, and analyze hotel booking data using Python, MySQL, Power BI and Tableau to derive meaningful insights.

Algorithm – Mobile Phone Sales Analysis:

1. Load the mobile sales dataset using Python.
2. Clean the data by handling missing values and duplicates.
3. Create the **Total Sales** field using Price × Quantity Sold.
4. Select 20,000 records and store them in SQL Server.
5. Retrieve and analyze the data using SQL queries.
6. Connect SQL Server data to Tableau.
7. Create charts for brands, regions, products, RAM, ROM, and sales trends.
8. Build an interactive Tableau dashboard and analyze the results.

PROGRAM:

```
import pandas as pd
import pyodbc
```

```
# =====
# 1. PYTHON – DATA CLEANING & TRANSFORMATION
# =====

# Load dataset
df = pd.read_excel(
    "cleaned_mobile_phone_sales_data.csv.xlsx"
)

# Create a copy for processing
```

```
mobile = df.copy()

# Remove duplicate records
mobile.drop_duplicates(inplace=True)

# Handle missing values
mobile["Brand"] = mobile["Brand"].fillna("Unknown")

mobile["Customer Location"] = (
    mobile["Customer Location"].fillna("Unknown")
)

mobile["Region"] = (
    mobile["Region"].fillna("Unknown")
)

# Convert numeric columns
mobile["Price"] = pd.to_numeric(
    mobile["Price"],
    errors="coerce"
)

mobile["Quantity Sold"] = pd.to_numeric(
    mobile["Quantity Sold"],
    errors="coerce"
)

# Convert date columns
mobile["Inward Date"] = pd.to_datetime(
    mobile["Inward Date"],
    errors="coerce"
)
```

```
mobile["Dispatch Date"] = pd.to_datetime(  
    mobile["Dispatch Date"],  
    errors="coerce"  
)
```

```
# Create derived column  
mobile["Total Sales"] = (  
    mobile["Price"] *  
    mobile["Quantity Sold"]  
)
```

```
# Select 20,000 records  
mobile = mobile.sample(  
    n=20000,  
    random_state=42  
)
```

```
print("Original Shape:", df.shape)  
print("Cleaned Shape:", mobile.shape)
```

```
# =====
```

```
# 2. SQL SERVER – DATABASE CONNECTION
```

```
# =====
```

```
server = "localhost"  
database = "MobileSalesProject"
```

```
connection_string = (  
    "DRIVER={ODBC Driver 17 for SQL Server};"  
    f"SERVER={server};"
```



```
f'DATABASE={database};"
"Trusted_Connection=yes;"
)

try:

    connection = pyodbc.connect(
        connection_string
    )

    print("\nConnected to SQL Server successfully!")

except Exception as e:

    print("\nSQL Server connection failed!")
    print(e)
    exit()

# =====
# 3. INSERT DATA INTO SQL SERVER
# =====

cursor = connection.cursor()

# Clear existing records
cursor.execute(
    "DELETE FROM MobileSales"
)

connection.commit()
```

```
# Insert query

insert_query = """
INSERT INTO MobileSales
(
    Product,
    Brand,
    [Product Code],
    [Product Specification],
    Price,
    [Inward Date],
    [Dispatch Date],
    [Quantity Sold],
    [Customer Name],
    [Customer Location],
    Region,
    [Processor Specification],
    RAM,
    ROM
)
VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
"""
```

```
# Insert records

for _, row in mobile.iterrows():

    cursor.execute(
        insert_query,

        row["Product"],
        row["Brand"],
        row["Product Code"],
        row["Product Specification"],
```

```

row["Price"]
if pd.isna(row["Price"])
else None,

row["Inward Date"]
if pd.isna(row["Inward Date"])
else None,

row["Dispatch Date"]
if pd.isna(row["Dispatch Date"])
else None,

row["Quantity Sold"]
if pd.isna(row["Quantity Sold"])
else None,

row["Customer Name"],
row["Customer Location"],
row["Region"],
row["Processor Specification"],
row["RAM"],
row["ROM"]
)

connection.commit()

print("Data loaded successfully into SQL Server!")

# =====
# 4. VERIFY DATA

```

```
# =====
```

```
cursor.execute(  
    "SELECT COUNT(*) FROM MobileSales"  
)
```

```
total_records = cursor.fetchone()[0]
```

```
print(  
    "\nTotal records in SQL Server:",  
    total_records  
)
```

```
# =====
```

```
# 5. TOTAL SALES ANALYSIS
```

```
# =====
```

```
print("\nTotal Sales")  
print("-----")
```

```
sales_query = """  
SELECT  
    SUM(Price * [Quantity Sold]) AS Total_Sales  
FROM MobileSales  
"""
```

```
sales_result = pd.read_sql(  
    sales_query,  
    connection  
)
```

```
print(sales_result)
```

```
# =====
```

```
# 6. TOTAL QUANTITY SOLD
```

```
# =====
```

```
print("\nTotal Quantity Sold")
```

```
print("-----")
```

```
quantity_query = """
```

```
SELECT
```

```
    SUM([Quantity Sold]) AS Total_Quantity
```

```
FROM MobileSales
```

```
"""
```

```
quantity_result = pd.read_sql(
```

```
    quantity_query,
```

```
    connection
```

```
)
```

```
print(quantity_result)
```

```
# =====
```

```
# 7. SALES BY BRAND
```

```
# =====
```

```
print("\nSales by Brand")
```

```
print("-----")
```

```
brand_query = """
```

```
SELECT
    Brand,
    SUM(Price * [Quantity Sold]) AS Total_Sales
FROM MobileSales
GROUP BY Brand
ORDER BY Total_Sales DESC
"""
```

```
brand_result = pd.read_sql(
    brand_query,
    connection
)
```

```
print(brand_result)
```

```
# =====
# 8. SALES BY REGION
# =====
```

```
print("\nSales by Region")
print("-----")
```

```
region_query = """
SELECT
    Region,
    SUM(Price * [Quantity Sold]) AS Total_Sales
FROM MobileSales
GROUP BY Region
ORDER BY Total_Sales DESC
"""
```

```
region_result = pd.read_sql(  
    region_query,  
    connection  
)
```

```
print(region_result)
```

```
# =====  
# 9. TOP 10 PRODUCTS  
# =====
```

```
print("\nTop 10 Products")  
print("-----")
```

```
product_query = """  
SELECT TOP 10  
    Product,  
    SUM([Quantity Sold]) AS Total_Quantity,  
    SUM(Price * [Quantity Sold]) AS Total_Sales  
FROM MobileSales  
GROUP BY Product  
ORDER BY Total_Sales DESC  
"""
```

```
product_result = pd.read_sql(  
    product_query,  
    connection  
)
```

```
print(product_result)
```

```
# =====
```

```
# 10. SALES BY RAM
```

```
# =====
```

```
print("\nSales by RAM")
```

```
print("-----")
```

```
ram_query = """
```

```
SELECT
```

```
    RAM,
```

```
    SUM(Price * [Quantity Sold]) AS Total_Sales
```

```
FROM MobileSales
```

```
GROUP BY RAM
```

```
ORDER BY Total_Sales DESC
```

```
"""
```

```
ram_result = pd.read_sql(
```

```
    ram_query,
```

```
    connection
```

```
)
```

```
print(ram_result)
```

```
# =====
```

```
# 11. SALES BY ROM
```

```
# =====
```

```
print("\nSales by ROM")
```

```
print("-----")
```



```
rom_query = """
SELECT
    ROM,
    SUM(Price * [Quantity Sold]) AS Total_Sales
FROM MobileSales
GROUP BY ROM
ORDER BY Total_Sales DESC
"""
```

```
rom_result = pd.read_sql(
    rom_query,
    connection
)
```

```
print(rom_result)
```

```
# =====
# 12. DATA RETRIEVAL FOR TABLEAU
# =====
```

```
tableau_query = """
SELECT
    Product,
    Brand,
    [Product Code],
    [Product Specification],
    Price,
    [Inward Date],
    [Dispatch Date],
    [Quantity Sold],
    [Customer Name],
```

```

[Customer Location],
Region,
[Processor Specification],
RAM,
ROM
FROM MobileSales
"""

tableau_data = pd.read_sql(
    tableau_query,
    connection
)

# Create Total Sales for Tableau analysis
tableau_data["Total Sales"] = (
    tableau_data["Price"] *
    tableau_data["Quantity Sold"]
)

print(
    "\nData retrieved for Tableau:",
    len(tableau_data),
    "records"
)

# =====
# 13. CLOSE SQL SERVER CONNECTION
# =====

cursor.close()
connection.close()

```

```
print("\nSQL Server connection closed.")
```

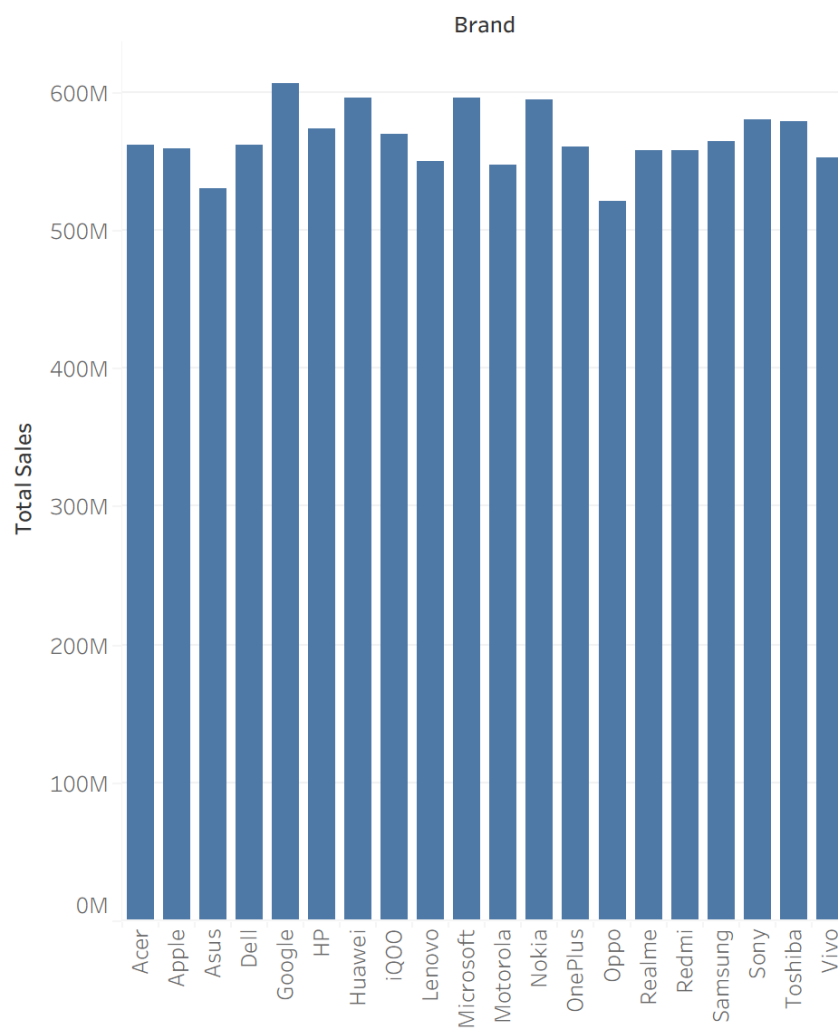
```
print("\n=====")
```

```
print("MOBILE SALES PROJECT COMPLETED")
```

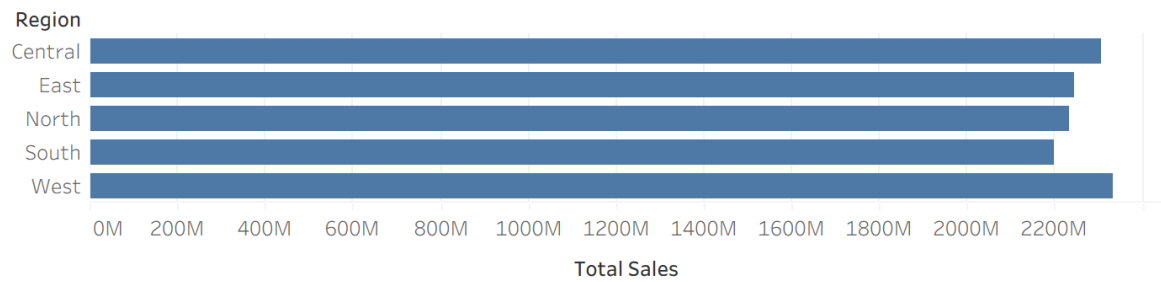
```
print("=====")
```

OUTPUT:

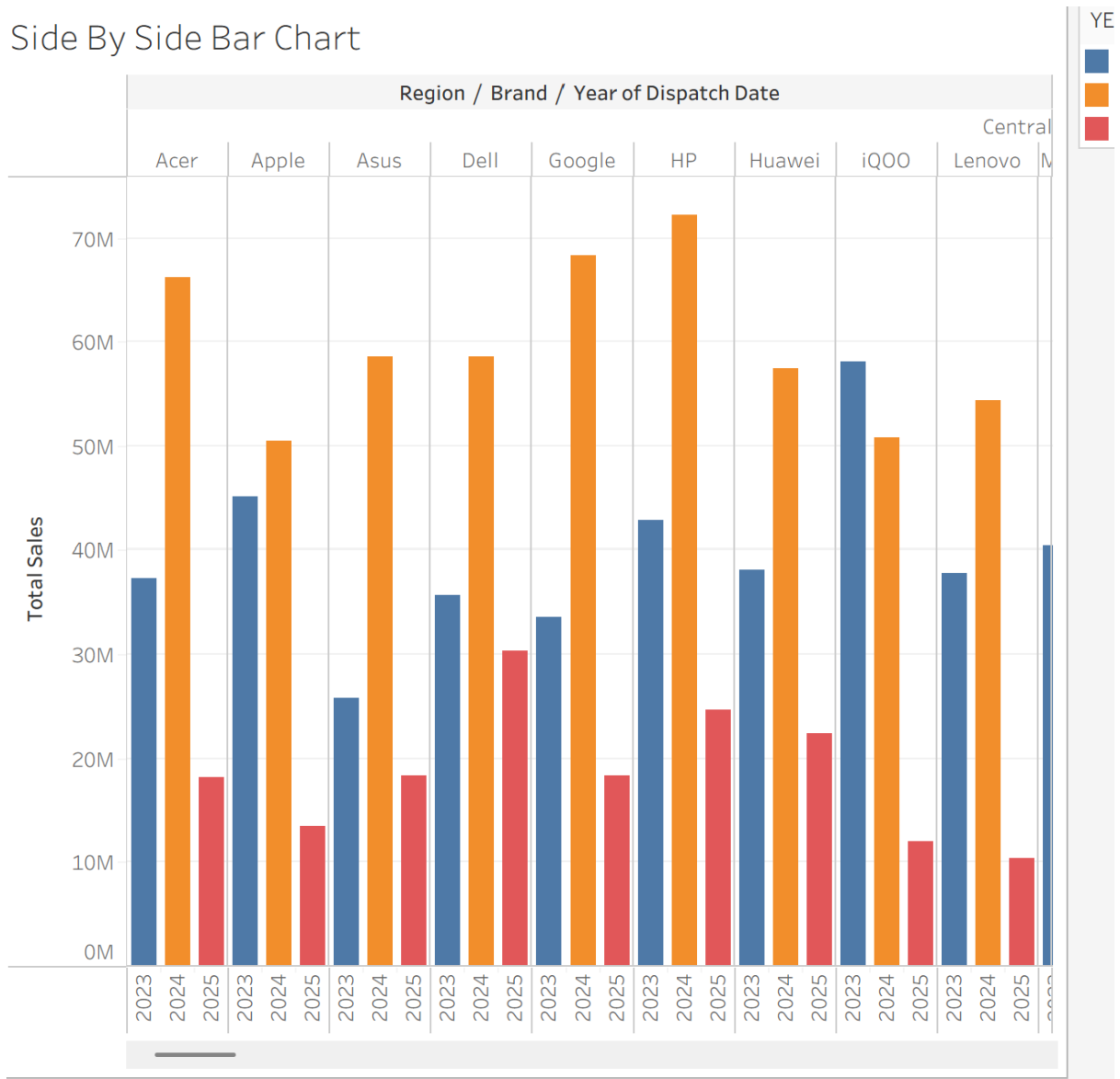
Sales by Brand



Sales by region



Side By Side Bar Chart



Result:

The Visualizations from tableau is done successfully.